

Java String Functions



string function with example in java



In Java, **String functions** are methods provided by the `String` class that allow you to manipulate and perform operations on strings. Here are some commonly used string functions with examples:

1. `length()`

Description: Returns the number of characters in the string.

```
java

public class StringExample {
    public static void main(String[] args) {
        String str = "Hello, Java!";
        System.out.println("Length: " + str.length()); // Output: Length: 12
    }
}
```

2. `charAt(index)`

Description: Returns the character at the specified index.

```
java

public class StringExample {
    public static void main(String[] args) {
        String str = "Hello, Java!";
        System.out.println("Character at index 1: " + str.charAt(1)); // Output: Character
at index 1: e
    }
}
```

3. `substring(beginIndex, endIndex)`

Description: Returns a substring from the specified range (endIndex is exclusive).

```
java

public class StringExample {
    public static void main(String[] args) {
        String str = "Hello, Java!";
        System.out.println("Substring: " + str.substring(7, 11)); // Output: Substring: Jav
```

```
a
    }
}
```

4. `toUpperCase()` and `toLowerCase()`

Description: Converts the string to upper case or lower case.

```
java

public class StringExample {
    public static void main(String[] args) {
        String str = "Hello, Java!";
        System.out.println("Upper case: " + str.toUpperCase()); // Output: Upper case: HELLO, JAVA!
        System.out.println("Lower case: " + str.toLowerCase()); // Output: Lower case: hello, java!
    }
}
```

5. `indexOf()`

Description: Returns the index of the first occurrence of a character or substring.

```
java

public class StringExample {
    public static void main(String[] args) {
        String str = "Hello, Java!";
        System.out.println("Index of 'J': " + str.indexOf('J')); // Output: Index of 'J': 7
        System.out.println("Index of 'Java': " + str.indexOf("Java")); // Output: Index of 'Java': 7
    }
}
```

6. `replace()`

Description: Replaces occurrences of a character or substring.

```
java

public class StringExample {
    public static void main(String[] args) {
        String str = "Hello, Java!";
        System.out.println("Replaced String: " + str.replace("Java", "World")); // Output: Replaced String: Hello, World!
    }
}
```

7. `equals()` and `equalsIgnoreCase()`

Description: Compares two strings for equality.

```
java

public class StringExample {
    public static void main(String[] args) {
        String str1 = "Hello, Java!";
        String str2 = "hello, java!";
        System.out.println("Equals: " + str1.equals(str2));           // Output: Equa
ls: false
        System.out.println("Equals Ignore Case: " + str1.equalsIgnoreCase(str2)); // Output:
Equals Ignore Case: true
    }
}
```

8. `trim()`

Description: Removes leading and trailing spaces.

```
java

public class StringExample {
    public static void main(String[] args) {
        String str = "  Hello, Java!  ";
        System.out.println("Trimmed String: '" + str.trim() + "'");
        // Output: Trimmed String: 'Hello, Java!'
    }
}
```

9. `split()`

Description: Splits the string into an array of substrings based on a delimiter.

```
java

public class StringExample {
    public static void main(String[] args) {
        String str = "Java,Python,C++";
        String[] languages = str.split(",");
        for (String lang : languages) {
            System.out.println(lang);
        }
        // Output:
        // Java
        // Python
        // C++
    }
}
```

10. `startsWith()` and `endsWith()`

Description: Checks if the string starts or ends with a specific prefix/suffix.

```
java
```

```
public class StringExample {
    public static void main(String[] args) {
        String str = "Hello, Java!";
        System.out.println("Starts with 'Hello': " + str.startsWith("Hello")); // Output: true
        System.out.println("Ends with 'Java!': " + str.endsWith("Java!")); // Output: true
    }
}
```

These are just a few examples of Java `String` functions. The `String` class has many other methods, and understanding these can greatly enhance your ability to work with text.



give me some other examples



Here are some additional examples of string functions in Java with explanations and code:

1. `contains()`

Description: Checks if the string contains a specific sequence of characters.

```
java
```

```
public class StringExample {
    public static void main(String[] args) {
        String str = "Welcome to Java programming!";
        System.out.println("Contains 'Java': " + str.contains("Java")); // Output: true
        System.out.println("Contains 'Python': " + str.contains("Python")); // Output: false
    }
}
```

2. `concat()`

Description: Concatenates (joins) two strings.

```
java
```

```
public class StringExample {
    public static void main(String[] args) {
        String str1 = "Hello, ";
        String str2 = "World!";
        System.out.println("Concatenated String: " + str1.concat(str2));
        // Output: Concatenated String: Hello, World!
    }
}
```

3. `isEmpty()`

Description: Checks if the string is empty (`""`).

```
java

public class StringExample {
    public static void main(String[] args) {
        String str1 = "";
        String str2 = "Java";
        System.out.println("Is str1 empty: " + str1.isEmpty()); // Output: true
        System.out.println("Is str2 empty: " + str2.isEmpty()); // Output: false
    }
}
```

4. `matches()`

Description: Tests if the string matches a given regular expression.

```
java

public class StringExample {
    public static void main(String[] args) {
        String str = "Java123";
        System.out.println("Matches '[a-zA-Z0-9]+' : " + str.matches("[a-zA-Z0-9]+"));
        // Output: true
        System.out.println("Matches '[0-9]+' : " + str.matches("[0-9]+"));
        // Output: false
    }
}
```

5. `intern()`

Description: Returns a string from the string pool if available, otherwise adds it to the pool.

```
java

public class StringExample {
    public static void main(String[] args) {
        String str1 = new String("Hello");
        String str2 = str1.intern();
        String str3 = "Hello";
        System.out.println("str2 == str3: " + (str2 == str3)); // Output: true
    }
}
```

6. `getBytes()`

Description: Encodes the string into a sequence of bytes using the default character set.

```
java
```

```
import java.util.Arrays;

public class StringExample {
    public static void main(String[] args) {
        String str = "Java";
        byte[] bytes = str.getBytes();
        System.out.println("Bytes: " + Arrays.toString(bytes));
        // Output: Bytes: [74, 97, 118, 97]
    }
}
```

7. `valueOf()`

Description: Converts data types to their string representation.

```
java
```

```
public class StringExample {
    public static void main(String[] args) {
        int num = 123;
        double d = 45.67;
        boolean b = true;
        String str = String.valueOf(num) + " " + String.valueOf(d) + " " + String.valueOf(b);
        System.out.println("String representation: " + str);
        // Output: String representation: 123 45.67 true
    }
}
```

8. `toCharArray()`

Description: Converts the string to a character array.

```
java
```

```
import java.util.Arrays;

public class StringExample {
    public static void main(String[] args) {
        String str = "Java";
        char[] chars = str.toCharArray();
        System.out.println("Character Array: " + Arrays.toString(chars));
        // Output: Character Array: [J, a, v, a]
    }
}
```

9. `lastIndexOf()`

Description: Returns the last occurrence of a character or substring.

```
java
```

```
public class StringExample {
    public static void main(String[] args) {
        String str = "Hello, Java!";
        System.out.println("Last index of 'a': " + str.lastIndexOf('a')); // Output: 10
    }
}
```

10. `repeat(int count)`

Description: Repeats the string a specified number of times.

(Available in Java 11 and later)

```
java
```

```
public class StringExample {
    public static void main(String[] args) {
        String str = "Java ";
        System.out.println("Repeated String: " + str.repeat(3));
        // Output: Java Java Java
    }
}
```

11. `strip()`, `stripLeading()`, and `stripTrailing()`

Description: Removes leading/trailing spaces, similar to `trim()` but with Unicode support.

(Available in Java 11 and later)

```
java
```

```
public class StringExample {
    public static void main(String[] args) {
        String str = " Hello, Java! ";
        System.out.println("Stripped: '" + str.strip() + "'"); // Output: 'Hello, Java!'
        System.out.println("Leading Stripped: '" + str.stripLeading() + "'"); // Output: 'Hello, Java!'
        System.out.println("Trailing Stripped: '" + str.stripTrailing() + "'"); // Output: 'Hello, Java!'
    }
}
```

These examples cover various operations like pattern matching, array conversion, and whitespace management. Let me know if you'd like more!



give more examples without repeating previously mentioned ones



Here are even more examples of Java string methods, avoiding repetitions from earlier examples:

1. `compareTo()`

Description: Compares two strings lexicographically.

- Returns 0 if equal, a negative value if the first string is lexicographically less, and a positive value otherwise.

```
java

public class StringExample {
    public static void main(String[] args) {
        String str1 = "Apple";
        String str2 = "Banana";
        System.out.println("Compare 'Apple' to 'Banana': " + str1.compareTo(str2));
        // Output: Compare 'Apple' to 'Banana': -1
    }
}
```

2. `compareToIgnoreCase()`

Description: Compares two strings lexicographically, ignoring case differences.

```
java

public class StringExample {
    public static void main(String[] args) {
        String str1 = "Apple";
        String str2 = "apple";
        System.out.println("Compare Ignore Case: " + str1.compareToIgnoreCase(str2));
        // Output: Compare Ignore Case: 0
    }
}
```

3. `copyValueOf(char[] data)`

Description: Returns a string that represents the characters of the specified char array.

```
java

public class StringExample {
    public static void main(String[] args) {
        char[] chars = { 'H', 'e', 'l', 'l', 'o' };
        String str = String.copyValueOf(chars);
        System.out.println("String from char array: " + str); // Output: Hello
    }
}
```

4. `regionMatches()`

Description: Tests if two string regions are equal.

```
java

public class StringExample {
    public static void main(String[] args) {
        String str1 = "Hello, Java!";
        String str2 = "Java";
        boolean result = str1.regionMatches(7, str2, 0, 4);
        System.out.println("Region Matches: " + result); // Output: true
    }
}
```

5. `replaceAll()`

Description: Replaces each substring matching a regular expression with a replacement string.

```
java

public class StringExample {
    public static void main(String[] args) {
        String str = "The quick brown fox jumps over the lazy dog";
        String result = str.replaceAll("fox", "cat");
        System.out.println("After replaceAll: " + result);
        // Output: The quick brown cat jumps over the lazy dog
    }
}
```

6. `replaceFirst()`

Description: Replaces the first substring matching a regular expression with a replacement string.

```
java

public class StringExample {
    public static void main(String[] args) {
        String str = "One One One";
        String result = str.replaceFirst("One", "Two");
        System.out.println("After replaceFirst: " + result);
        // Output: Two One One
    }
}
```

7. `chars()`

Description: Returns an `IntStream` of the characters in the string.

(Available in Java 8 and later)

```
java

import java.util.stream.Collectors;
```

```
public class StringExample {
    public static void main(String[] args) {
        String str = "Hello";
        String result = str.chars()
            .mapToObj(c -> (char) c)
            .collect(Collectors.toList())
            .toString();
        System.out.println("Characters in the string: " + result);
        // Output: Characters in the string: [H, e, l, l, o]
    }
}
```

8. `codePointAt()`

Description: Returns the code point at the specified index.

```
java

public class StringExample {
    public static void main(String[] args) {
        String str = "Hello";
        int codePoint = str.codePointAt(0);
        System.out.println("Code point at index 0: " + codePoint);
        // Output: 72
    }
}
```

9. `codePointCount()`

Description: Returns the number of Unicode code points in a text range.

```
java

public class StringExample {
    public static void main(String[] args) {
        String str = "Hello";
        int codePoints = str.codePointCount(0, str.length());
        System.out.println("Code point count: " + codePoints);
        // Output: 5
    }
}
```

10. `split(regex, limit)`

Description: Splits a string into a limited number of substrings based on a regex.

```
java

public class StringExample {
    public static void main(String[] args) {
        String str = "a,b,c,d,e";
        String[] parts = str.split(",", 3);
        for (String part : parts) {
            System.out.println(part);
        }
    }
}
```

```
// Output:
// a
// b
// c,d,e
}
}
```

11. `join(CharSequence delimiter, CharSequence... elements)`

Description: Joins multiple strings with a specified delimiter.

(Available in Java 8 and later)

```
java

public class StringExample {
    public static void main(String[] args) {
        String joined = String.join(", ", "Apple", "Banana", "Cherry");
        System.out.println("Joined String: " + joined);
        // Output: Joined String: Apple, Banana, Cherry
    }
}
```

12. `lines()`

Description: Splits the string into lines and returns a Stream.

(Available in Java 11 and later)

```
java

import java.util.stream.Stream;

public class StringExample {
    public static void main(String[] args) {
        String str = "Line 1\nLine 2\nLine 3";
        Stream<String> lines = str.lines();
        lines.forEach(System.out::println);
        // Output:
        // Line 1
        // Line 2
        // Line 3
    }
}
```

13. `formatted()`

Description: Returns a formatted string, similar to `String.format()`.

(Available in Java 15 and later)

```
java

public class StringExample {
    public static void main(String[] args) {
        String str = "Hello, %s!";
    }
}
```

```
System.out.println(str.formatted("Java"));
// Output: Hello, Java!
    }
}
```

These examples should add more depth to your knowledge of Java string methods! Let me know if you want an explanation of any specific method.



give me any other examples if available besides these



Here are some additional, less commonly discussed `String` methods or features not covered yet, offering more examples:

1. `codePoints()`

Description: Returns an `IntStream` of Unicode code points in the string.

(Available in Java 8 and later)

```
java

import java.util.stream.Collectors;

public class StringExample {
    public static void main(String[] args) {
        String str = "Hello, Java!";
        String codePoints = str.codePoints()
                                .mapToObj(Integer::toString)
                                .collect(Collectors.joining(", "));
        System.out.println("Code points: " + codePoints);
        // Output: Code points: 72, 101, 108, 108, 111, 44, 32, 74, 97, 118, 97, 33
    }
}
```

2. `indent(int n)`

Description: Adjusts the indentation of each line in the string by the given number of spaces.

- Positive `n`: Adds spaces.
- Negative `n`: Removes spaces if present.

(Available in Java 12 and later)

```
java

public class StringExample {
    public static void main(String[] args) {
        String str = "Hello\nWorld";
        System.out.println("Indented String:\n" + str.indent(4));
        System.out.println("Reduced Indentation:\n" + str.indent(-2));
    }
}
```

```
}
}
```

3. `resolveConstantDesc()`

Description: Resolves the constant description for use with the `ConstantDesc` API.

(Available in Java 12 and later)

```
java

import java.lang.constant.ConstantDesc;

public class StringExample {
    public static void main(String[] args) {
        String str = "Java";
        ConstantDesc constantDesc = str.resolveConstantDesc(null);
        System.out.println("Constant Description: " + constantDesc);
    }
}
```

4. `repeat(n)`

Description: Creates a new string by repeating the original string `n` times.

(Available in Java 11 and later)

```
java

public class StringExample {
    public static void main(String[] args) {
        String str = "Hi! ";
        System.out.println("Repeated String: " + str.repeat(3));
        // Output: Hi! Hi! Hi!
    }
}
```

5. `translateEscapes()`

Description: Replaces escape sequences in a string with their literal equivalents.

(Available in Java 15 and later)

```
java

public class StringExample {
    public static void main(String[] args) {
        String str = "Hello\\nWorld";
        String translated = str.translateEscapes();
        System.out.println("Translated: " + translated);
        // Output:
        // Hello
        // World
    }
}
```

```
}
}
```

6. `stripIndent()`

Description: Strips leading spaces common to all lines in the string.

(Available in Java 13 and later)

```
java

public class StringExample {
    public static void main(String[] args) {
        String str = "    Hello\n        Java";
        System.out.println("Before Strip Indent:\n" + str);
        System.out.println("After Strip Indent:\n" + str.stripIndent());
    }
}
```

7. `isBlank()`

Description: Checks if a string is empty or contains only whitespace.

(Available in Java 11 and later)

```
java

public class StringExample {
    public static void main(String[] args) {
        String str1 = " ";
        String str2 = "Java";
        System.out.println("Is str1 blank: " + str1.isBlank()); // Output: true
        System.out.println("Is str2 blank: " + str2.isBlank()); // Output: false
    }
}
```

8. `transform(Function<? super String, ? extends String>)`

Description: Transforms the string using a specified function and returns the result.

(Available in Java 12 and later)

```
java

public class StringExample {
    public static void main(String[] args) {
        String str = "java";
        String result = str.transform(s -> s.toUpperCase());
        System.out.println("Transformed String: " + result); // Output: JAVA
    }
}
```

9. `formatted(Object... args)`

Description: A shorthand method to format a string using placeholder substitutions.
(Available in Java 15 and later)

```
java

public class StringExample {
    public static void main(String[] args) {
        String template = "Name: %s, Age: %d";
        String result = template.formatted("Alice", 25);
        System.out.println(result); // Output: Name: Alice, Age: 25
    }
}
```

10. `hashCode()`

Description: Returns the hash code of the string.

```
java

public class StringExample {
    public static void main(String[] args) {
        String str = "Java";
        System.out.println("HashCode: " + str.hashCode());
        // Output: HashCode: [Some integer value based on the string]
    }
}
```

11. `splitAsStream()`

(via `Pattern` class)

Description: Splits the string and returns a stream of the parts.

```
java

import java.util.regex.Pattern;

public class StringExample {
    public static void main(String[] args) {
        String str = "apple,banana,orange";
        Pattern.compile(",").splitAsStream(str).forEach(System.out::println);
        // Output:
        // apple
        // banana
        // orange
    }
}
```

12. `joinWith()` (using `Stream API`)

Description: Concatenates strings with a delimiter using streams.

java

```
import java.util.List;

public class StringExample {
    public static void main(String[] args) {
        List<String> items = List.of("Dog", "Cat", "Rabbit");
        String result = String.join(", ", items);
        System.out.println("Joined String: " + result);
        // Output: Joined String: Dog, Cat, Rabbit
    }
}
```

These additional examples extend into newer Java versions, showing functions that utilize modern APIs and Unicode handling techniques. Let me know if you'd like help with any specific use case or concept!